Rapport du Projet : Groupe 20

1. Introduction

Ce projet implémente un système complet de gestion hospitalière, permettant la gestion des patients, du personnel médical, des visites et des hospitalisations. Le système repose sur une architecture moderne, combinant **MySQL** pour la persistance des données et **Python** pour l'interface utilisateur. L'objectif est de fournir un outil robuste, fonctionnel et automatisé, adapté aux besoins d'un établissement de santé.

Afin de garantir une conception structurée et compréhensible, nous avons utilisé l'outil **DB-Main** pour modéliser notre base de données selon deux approches complémentaires : un **Modèle Conceptuel de Données (MCD)** et un **schéma relationnel**. Ces deux modèles ont guidé la construction logique et physique du système, depuis la définition des entités et de leurs relations jusqu'à l'implémentation en SQL.

2. Architecture Technique

2.1 Base de Données (`BD_Groupe20.sql`)

Structure des Tables

- **PERSONNEL (Table principale)**
  Attributs : `num_personnel` (PK), `telephone`, `date_embauche`, `e_mail`, `prenom`, `nom`, `NOM_DEP` (FK)
  Rôle : Stockage centralisé des informations du personnel hospitalier
  Contraintes : Clé étrangère vers la table `DEPARTEMENT`
- **MEDECIN (Spécialisation de PERSONNEL)**
  Attributs : `num_personnel` (PK/FK), `Type`, `Disponibilite`

Contraintes : Type limité à `'Hospitalisation'` ou `'Consultation'`
Relations : Héritage de `PERSONNEL`

- **INFIRMIERE (Spécialisation de PERSONNEL)**
  Attributs : `num_personnel` (PK/FK), `Disponibilite`
  Relations : Héritage de `PERSONNEL`
- **DEPARTEMENT**
  Attributs : `NOM_DEP` (PK), `CHEF`
  Relations : Lié à `PERSONNEL` et `VISITE`
- **PATIENT**
  Attributs : `id_patient` (PK), `nom`, `prenom`, `sexe`, `adresse`, `e_mail`
  Relations : Lié aux `VISITE`
- **VISITE**
  Attributs : `num_visite` (PK), `duree`, `date_visite`, `time_of_visite`, `motif`, `id_patient` (FK), `num_chambre` (FK), `NOM_DEP` (FK)
  Relations : Centre névralgique liant `PATIENT`, `CHAMBRE` et `DEPARTEMENT`
- **HOSPITALISATION**
  Attributs : `id_hospitalisation` (PK), `type_hospitalisation`, `num_visite` (FK)
  Relations : Liée à `VISITE` et `INFIRMIERE` via la table `PARTICIPE`
- **CONSULTATION**
  Attributs : `id_consultation` (PK), `num_visite` (FK)
  Relations : Liée à `VISITE` et `ANALYSE_MEDICAL`
- **ANALYSE_MEDICAL**
  Attributs : `num_analyse` (PK), `nom_analyse`, `nom_médicament`, `id_consultation` (FK)
  Relations : Liée à `CONSULTATION`
- **CHAMBRE**
  Attributs : `num_chambre` (PK), `Surface`, `Type`, `Nombre_de_lits`, `Occupation`
  Contraintes : Type limité à `'Bloc Hospitalisation'` ou `'Bloc Consultation'`
- **PARTICIPE (Table de liaison)**
  Attributs : `id_hospitalisation` (PK/FK), `num_personnel` (PK/FK)
  Relations : Lie `INFIRMIERE` et `HOSPITALISATION`
- **EFFECTUE (Table de liaison)**
  Attributs : `num_visite` (PK/FK), `num_personnel` (PK/FK)
  Relations : Lie `PERSONNEL` et `VISITE`

Vues SQL Implémentées

- **vue_patient_visite**
  Objectif : Historique des visites par patient
  Champs : Informations du patient et détails des visites
- **vue_visites_medecins**
  Objectif : Suivi des consultations et hospitalisations
  Inclut : Type de visite, informations du patient et du médecin
- **vue_hospitalisation_infirmiere**
  Objectif : Suivi des affectations infirmières
  Détails : Liens entre hospitalisations et personnel infirmier
- **vue_medecins_disponibles**
  Objectif : Liste des médecins disponibles
  Filtrage : Uniquement médecins avec `disponibilite = TRUE`

- **vue_occupation_chambres**
  Objectif : État d'occupation des chambres
  Monitoring : Capacité et occupation en temps réel

Triggers

- **trg_insert_consultation_or_hospitalisation**
  Déclenchement : Après insertion dans `VISITE`
  Action : Création automatique d'une `CONSULTATION` ou d'une `HOSPITALISATION` selon le motif
- **trg_participe_after_hospitalisation**
  Déclenchement : Après création d'une `HOSPITALISATION`
  Action : Affectation automatique d'une infirmière disponible

2.2 Application Python (`Groupe20.py`)

2.2.1 Structure Principale

L'application propose **trois options principales** accessibles via un **menu interactif** :

- `'N'` : **Nouveau patient**
- `'E'` : **Embauche de personnel**
- `'A'` : **Autres opérations** (historique et consultation des tables)

2.2.2 Fonctionnalités Implémentées

Gestion des Patients

- Enregistrement des informations de base : `nom`, `prénom`, `sexe`, `adresse`, `email`
- Création automatique d'une **visite associée**
- Attribution d'une **chambre** selon le **type de visite** (`consultation` ou `hospitalisation`)
- Durée prédéfinie :
  - **20 minutes** pour une consultation
  - **60 minutes** pour une hospitalisation

Gestion du Personnel

- **Embauche de médecins** avec :
  - Spécification du **type** : `Consultation` ou `Hospitalisation`
  - Choix du **département d'affectation**
  - Saisie des **informations personnelles**
- **Embauche d'infirmières** avec :
  - Affectation directe à un **département**
  - Gestion de leur **disponibilité**

Consultation des Données

- Affichage de l'**historique des visites** d'un patient
- Visualisation du contenu des tables suivantes :
  - `EFFECTUE`
  - `PARTICIPE`
  - `ANALYSE_MEDICAL`
  - `CONSULTATION`
  - `HOSPITALISATION`
  - `VISITE`
  - `INFIRMIERE`
  - `MEDECIN`
  - `PERSONNEL`
  - `DEPARTEMENT`
  - `CHAMBRE`
  - `PATIENT`

2.2.3 Connexion à la Base de Données

- Utilisation de la bibliothèque **`mysql.connector`**
- **Paramètres de connexion** :
  - Utilisateur : `youssef`
  - Base de données : `hospital_database`
- Gestion des **erreurs de connexion** incluse

2.3 Conteneurisation (Docker)

La conteneurisation de notre application hospitalière utilise **Docker** pour garantir un **déploiement cohérent et isolé**. Cette approche moderne de déploiement offre plusieurs avantages cruciaux pour notre système :

2.3.1 Architecture des Conteneurs

Notre solution repose sur une architecture à **deux conteneurs interconnectés** :

Conteneur Base de Données

- Basé sur **MySQL 8.0**
- Garantit l'**isolation des données**
- Configuration automatique de la base de données hospitalière
- Gestion sécurisée des identifiants et mots de passe
- **Persistance des données** via des volumes dédiés

Conteneur Application

- Environnement **Python optimisé et léger**
- Isolation complète des dépendances
- Communication sécurisée avec la base de données
- Interface utilisateur en mode terminal interactive

2.3.2 Avantages de la Conteneurisation

Portabilité

- Déploiement uniforme sur différents environnements
- Élimination des problèmes de compatibilité
- Migration facilitée entre les environnements

Reproductibilité

- Environnement de développement identique pour tous
- Tests cohérents et fiables
- Déploiement standardisé

Isolation

- Séparation claire entre l'application et la base de données
- Sécurité renforcée
- Gestion indépendante des ressources

Maintenance Simplifiée

- Mises à jour facilitées des composants
- Sauvegarde simplifiée des données
- Gestion efficace des versions

2.3.3 Organisation des Services

- **Orchestration** : Utilisation de Docker Compose pour la gestion multi-conteneurs
- **Réseau** : Création automatique d'un réseau isolé entre les services
- **Volumes** : Persistance des données de la base MySQL
- **Configuration** : Initialisation automatique de la base de données

2.3.4 Sécurité

- Isolation complète des services
- Gestion sécurisée des variables d'environnement
- Restriction des accès réseau
- Protection des données sensibles

Cette architecture conteneurisée assure un **déploiement robuste et sécurisé** de notre système de gestion hospitalière, tout en facilitant sa **maintenance** et son **évolution future**.

3. Sécurité et Intégrité des Données

- Gestion des utilisateurs MySQL avec des privilèges limités
- Contraintes d'intégrité référentielle
- Validation des données en entrée
- Utilisation de transactions sécurisées

4. Interface Utilisateur

Un menu interactif est disponible, offrant les fonctionnalités suivantes :

- Enregistrement de nouveaux patients
- Gestion du personnel médical
- Consultation des données
- Génération de rapports

5. Points Forts du Projet

- Architecture modulaire et extensible
- Gestion automatisée des processus métier
- Interface utilisateur intuitive
- Déploiement simplifié via Docker
- Utilisation de vues SQL pour optimiser les requêtes complexes

6. Présentation des Modèles de la Base de Données Hospitalière

Notre système de gestion hospitalière est représenté à travers **deux modèles complémentaires** qui offrent des perspectives différentes mais interconnectées de la base de données.

Analyse du Schéma Relationnel

Le **schéma relationnel**, composé de **12 tables interconnectées**, représente la **structure physique** de notre base de données.

Structure Centrale du Personnel

- La table `PERSONNEL` forme l'épine dorsale du système, stockant les informations essentielles des employés (`num_personnel`, `téléphone`, `date_embauche`, etc.).
- Une hiérarchie sophistiquée est mise en place via les tables `MEDECIN` et `INFIRMIERE` qui héritent de `PERSONNEL`, permettant une spécialisation précise des rôles.
- Chaque membre du personnel est rattaché à un `DEPARTEMENT`, assurant une organisation structurée des services.

Gestion des Patients et Visites

- La table `PATIENT` centralise les données personnelles des patients.

- La table `VISITE` joue un rôle pivot, enregistrant chaque interaction patient-hôpital avec des attributs détaillés (`date_visite`, `durée`, `motif`).
- Un système de suivi complet est implémenté via les tables `HOSPITALISATION` et `CONSULTATION`, permettant de tracer différents types de prise en charge.

Optimisation des Ressources

- La table `CHAMBRE` gère l'infrastructure physique avec des attributs précis (`Surface`, `Type`, `Nombre_de_lits`).
- Les tables de liaison `PARTICIPE` et `EFFECTUE` permettent une gestion flexible des affectations du personnel.

Analyse du Modèle Conceptuel de Données (MCD)

Le **MCD** offre une **vision métier** plus abstraite et plus facile à comprendre pour les non-techniciens.

Organisation Hiérarchique

- Représentation claire de la généralisation/spécialisation entre `PERSONNEL`, `MEDECIN`, et `INFIRMIERE` via un triangle d'héritage.
- Visualisation immédiate des responsabilités via les cardinalités (ex : un médecin peut effectuer plusieurs visites).

Flux Patient

- Le modèle met en évidence le parcours patient à travers différentes entités.
- Une relation de disjonction `'P'` sophistiquée permet de différencier les visites aboutissant soit à une `HOSPITALISATION`, soit à une `CONSULTATION`.
- Les cardinalités précises (1–N, 0–N) définissent clairement les règles métier.

Intégration des Services

- Représentation claire des interactions entre `DEPARTEMENT` et autres entités.
- Visualisation immédiate des liens entre `PERSONNEL`, `VISITES` et services associés

Complémentarité des Modèles

Les deux schémas se **complètent parfaitement** :

- Le **MCD** offre une **vision stratégique et conceptuelle**, idéale pour la planification et la communication avec les parties prenantes.
- Le **schéma relationnel** fournit le **blueprint technique** nécessaire à l'implémentation physique dans le SGBD.

Cette double approche garantit une **conception robuste** et une **implémentation fidèle aux besoins métiers de l'hôpital**.

6. Conclusion

Le système répond aux besoins de gestion hospitalière grâce à une architecture robuste, une base de données bien structurée et une interface fonctionnelle. La conteneurisation assure un déploiement rapide, cohérent et reproductible.